Cómo funciona una función en Python

Week 01 - Logic | Class 04 - Functions | Companion note for EX10

Mapa mental

Idea central: definir una función crea una herramienta; llamarla ejecuta esa herramienta con datos concretos.

1. Qué ocurre cuando Python encuentra `def`

Al ejecutar una línea `def`, Python crea un objeto función y asocia ese objeto al nombre de la función. El cuerpo todavía no se ejecuta.

```
def calculate_rocket_delta_v(isp, m0, mf, g0=9.80665):
    ...
```

Después de esta definición, Python sabe que existe una función llamada calculate_rocket_delta_v. El cálculo del delta-v solo ocurre cuando la llamamos.

2. Parámetros y argumentos

Parámetros son los nombres que aparecen en la definición. Argumentos son los valores u objetos que entregamos en una llamada.

Definición	Llamada	Qué ocurre
isp	250	isp recibe 250
m0	500000	m0 recibe 500000
mf	250000	mf recibe 250000
g0=9.80665	(omitido)	se usa el valor por defecto

3. Tu observación sobre los nombres: posicional vs keyword

Aquí está el matiz importante: en Python los argumentos posicionales NO necesitan tener el mismo nombre que los parámetros. Solo importa el orden.

```
engine_isp = 250
wet_mass = 500000
dry_mass = 250000

dv = calculate_rocket_delta_v(engine_isp, wet_mass, dry_mass)
```

Esto funciona aunque las variables externas se llamen `engine_isp`, `wet_mass` y `dry_mass`. Python enlaza por posición: primero con `isp`, segundo con `m0`, tercero con `mf`.

En cambio, con keyword arguments, el nombre de la izquierda sí debe coincidir exactamente con el parámetro de la definición:

```
dv = calculate_rocket_delta_v(
    isp=engine_isp,
    m0=wet_mass,
    mf=dry_mass
)
```

Regla práctica

En una llamada keyword, isp=, m0= y mf= pertenecen a la interfaz de la función. A la derecha del = puedes usar cualquier variable o expresión.

4. ¿Y en C/C++?

Tu recuerdo es correcto. En C y C++, las llamadas normales son posicionales: los nombres usados por el código que llama son irrelevantes. El compilador enlaza los argumentos por orden y tipo. Los nombres de los parámetros no forman parte de la firma de la función.

```
// C++
double rocketDv(double isp, double m0, double mf);

double engineIsp = 250;
double wetMass = 500000;
double dryMass = 250000;

double dv = rocketDv(engineIsp, wetMass, dryMass);
```

Python también permite esta forma posicional, pero añade una capacidad extra muy útil: llamar por nombre mediante keyword arguments.

5. Qué ocurre exactamente durante una llamada

1	Python evalúa los argumentos de la llamada.
2	Los enlaza con los parámetros: por posición, por keyword o usando defaults.
3	Crea un nuevo contexto local para esa llamada (un frame).
4	Ejecuta el cuerpo de la función de arriba abajo.
5	Si encuentra return, termina inmediatamente la función.
6	El objeto devuelto vuelve al punto exacto donde se hizo la llamada.
7	La variable del programa principal puede guardar ese objeto.

```
delta_v = calculate_rocket_delta_v(250, 500000, 250000)
```

Mentalmente puedes leerlo como:

delta_v <- resultado devuelto por calculate_rocket_delta_v(...)

6. Scope: parámetros y variables locales

Dentro de la función, `isp`, `m0`, `mf`, `mass_ratio` y `delta_v` son locales a esa llamada. Una nueva llamada crea un nuevo conjunto de valores locales.

> Reutilización
> Por eso puedes reutilizar la misma función cientos de veces: cada llamada tiene su propio estado local.

7. Return, return None y salida inmediata

```
if mf <= 0:
    return None

mass_ratio = m0 / mf
```

Si `mf <= 0`, Python ejecuta `return None` y sale inmediatamente. Las líneas posteriores de esa llamada no se ejecutan. El programa principal recibe `None` y puede decidir qué hacer.

8. Parámetros por defecto

```
def calculate_rocket_delta_v(isp, m0, mf, g0=9.80665):
```

Los tres primeros parámetros son obligatorios. `g0` es opcional porque tiene un valor por defecto. Si se omite, se usa 9.80665; si se proporciona, el argumento suministrado sustituye al default.

9. Reglas útiles para llamadas

Regla	Significado
Posicional	Se enlaza por orden. Los nombres externos no importan.
Keyword	El nombre a la izquierda del = debe coincidir con el parámetro.
Default	Se usa solo si ese parámetro no recibe otro argumento.
Mezcla	Los argumentos posicionales deben aparecer antes que los keyword.
Duplicado	No puedes proporcionar el mismo parámetro dos veces.
Mayúsculas	Python distingue isp de Isp: son nombres diferentes.

10. Revisión de tu EX10

Tu ejercicio demuestra correctamente llamadas posicionales, keyword arguments, un default para g0, validaciones tempranas con return None y reutilización de la función.

Requisito del modelo

Detalle a decidir: tu función rechaza `m0 == mf` porque usa `m0 <= mf`. Si quieres que el caso sin consumo de propelente sea matemáticamente válido, cambia a `m0 < mf`; entonces `m0 == mf` devolverá delta-v = 0. Si prefieres considerar que no es un burn válido, tu código actual es coherente, pero documenta esa decisión.

Para escalares, `math.log` es suficiente y evita depender de NumPy. `np.log` no está mal; será especialmente útil cuando trabajemos con arrays.

11. Learning Tracker

Concepto	Qué debes recordar
Function definition	def crea una función; no ejecuta el cuerpo.
Parameters	Nombres internos que reciben valores.
Arguments	Valores entregados en la llamada.
Positional binding	Enlace por orden.
Keyword binding	Enlace por nombre.
Default parameter	Valor utilizado si se omite el argumento.
Local scope	Cada llamada crea sus propias variables locales.
return	Termina la función y devuelve un objeto.
return None	Salida temprana sin resultado numérico válido.
Reuse	La misma función puede usarse con muchos casos.

Pregunta de control: si ves f(a=external_value), ¿qué nombre debe coincidir con el parámetro de la función? Respuesta: a. El nombre external_value puede ser cualquiera.